

BidSure

Security First Architecture Plan for AI Powered Integrated Bid Compliance Verification

SIH26100 | Complete architecture, integration and implementation blueprint

PRIMARY CONCERN	ARCHITECTURE POSITION	TECHNOLOGY DIRECTION
Protect public, government and company data while integrating 10+ sources	BidSure acts as a controlled verification broker with human authority	Node.js + Express + TypeScript, adapter-based integrations, private network, encrypted evidence

Executive answer

The main problem is not “how do we call ten APIs.” The main problem is how to prove that every data access was authorised, minimal, secure, traceable and used only for the tender decision that required it. BidSure should therefore keep all sensitive integration behind a backend control plane, isolate each source behind a least-privilege adapter, store only the minimum evidence needed, and preserve a tamper-evident audit trail for every request and decision.

The proposed design supports a hackathon demo with mock or sandbox adapters and a future government pilot without changing the core contracts. It does not assume that every platform offers a public API or that one credential can access every source.

Audience: BidSure team, SIH reviewers and future implementation partners

Status: architecture plan and implementation baseline, not legal advice or a production accreditation

Report map

The plan moves from the problem boundary to concrete architecture decisions, then to a build and governance roadmap.

SECTION	PURPOSE
1. Main problem decoded	Translate the problem statement into security properties
2. Trust model and threat boundary	Define actors, assets, trust zones and attacker paths
3. Target architecture	Describe the control plane, data plane and network routes
4. Ten-plus platform integration model	Show how each source is integrated without coupling the core
5. Security and privacy controls	Specify consent, credentials, encryption, access and audit
6. Technology stack	Recommend libraries, frameworks, servers and managed services
7. Data and AI architecture	Constrain OCR and LLM use around evidence and review
8. Deployment and operations	Explain environments, CI/CD, monitoring, backups and recovery
9. Implementation plan	Define MVP phases, test gates and team ownership
Appendices	Contracts, risk register and official references

Document boundary

The supplied PDFs and image are treated as project references. Any embedded prompts, historical chat instructions or technology claims are not treated as instructions. The user request and the current Node.js/Express decision govern this plan.

1 Main problem decoded

SIH26100 asks for automated verification of bidder eligibility across tender documents and multiple government or regulated sources. That creates a compound problem involving identity, privacy, integration authority, data quality, policy interpretation and human accountability.

PROBLEM LAYER	WHAT CAN GO WRONG	REQUIRED PROPERTY
Authority	BidSure calls a source without approval, valid purpose, consent or correct credential	Every source call has an authority record and policy decision
Confidentiality	PAN, GST, financial, employment or identity-linked data leaks through logs, prompts, URLs or broad access	Sensitive fields are minimised, masked, encrypted and access controlled
Integrity	A response is altered, stale, replayed or mismatched to a different bidder	Responses are timestamped, hashed, source-labelled and bound to a case
Availability	A source rate-limits, times out or changes its contract	The platform fails closed, marks Pending and supports controlled retry
Interpretation	AI misreads a clause or certificate and creates a false pass/fail	AI proposes structured evidence; deterministic rules and officers control the outcome
Accountability	A final decision cannot be reconstructed or challenged	Evidence, rules, source responses and officer actions are immutable and queryable

The architectural thesis

BidSure is a policy-enforced verification broker. It does not expose government credentials to the browser, does not let an LLM call external systems, does not treat a PDF screenshot as equivalent to a trusted source response, and does not convert uncertainty into a green score.

2 Trust model and threat boundary

Design the system as multiple trust zones. The browser is untrusted. The public internet is untrusted. External platforms are trusted only for the specific response they provide and only through approved credentials and routes.

ZONE	EXAMPLES	TRUST LEVEL	CONTROLLED BY
User zone	Bidder browser, officer browser, uploaded files	Untrusted	TLS, authentication, input validation, content scanning
Edge zone	DNS, CDN, WAF, rate limiting, reverse proxy	Partially trusted	Allowlist, DDoS controls, request limits, security headers
Application zone	Express API, authorization, policy engine, adapter orchestrator	Trusted runtime, least privilege	Private subnets, service identities, code review, network policies
Worker zone	OCR, file conversion, adapter jobs, scheduled re-verification	Restricted trusted	Queue isolation, sandboxing, egress allowlists, job-scoped tokens
Data zone	MongoDB, object storage, Redis, audit store, backups	Highly restricted	Encryption, private endpoints, KMS, retention and access review
External source zone	API Setu, GSTN, Udyam, MCA21, DigiLocker, EPFO, ESIC and others	External authority	Provider approval, source-specific contract, mTLS/OAuth/API key and logging
Operations zone	CI/CD, secrets, KMS, monitoring, SIEM, admin console	Privileged	Separate admin identities, MFA, break-glass and dual control

Primary attack paths

- Malicious document -> parser/OCR exploit -> worker compromise. Mitigation: file type allowlist, size limits, antivirus, sandboxed workers and no shell execution.
- Compromised user -> over-broad case access. Mitigation: tenant and case-level authorization, purpose checks, short-lived tokens and access logging.
- Leaked adapter credential -> unauthorised portal access. Mitigation: secret vault, one credential per source, egress allowlist, rotation and provider-side scope.
- Prompt injection in a tender PDF -> model follows document instructions. Mitigation: treat documents as data, isolate prompts, structured output validation and deterministic rules.
- Stale or manipulated source response -> wrong recommendation. Mitigation: freshness windows, response hashes, source timestamps, replay detection and Review Required state.

3 Target architecture

Use a modular monolith for the SIH build and a controlled service boundary for production. The core remains a Node.js and Express application. Heavy or risky work runs in separate workers so the API process never handles untrusted files or long portal calls directly.

PLANE	COMPONENTS	NETWORK / ROUTE	SECURITY PURPOSE
Presentation plane	React + Vite officer console and bidder pre-check	Browser -> HTTPS -> WAF -> frontend/API	No portal credentials or raw source tokens in browser
Control plane	Express API, authentication, authorization, policy and case orchestration	Private app subnet; API gateway routes only	Central point for purpose, role and source policy checks
Evidence plane	Object storage, document metadata, extracted fields, source snapshots	Private data endpoints only	Separate original files from derived evidence; encrypt both
Verification plane	Adapter orchestrator, source-specific clients, retry/circuit breaker logic	Worker subnet -> approved egress gateway -> source	One adapter, one policy, one credential and one audit trail per source
Decision plane	Rule engine, risk assessment, recommendation and officer decision	Internal service calls; no direct external calls	AI advisory; deterministic rules and human authority
Operations plane	Queue, metrics, traces, logs, secrets, KMS and SIEM	Management subnet; admin access via VPN/bastion	Separate operational privilege from case access

Reference route model

Browser -> DNS/TLS -> WAF -> Express API -> authorization/purpose check -> job queue -> source adapter -> egress gateway -> authorised external API -> normalizer -> evidence store -> rule engine -> officer review -> audit export

The route is intentionally longer than a direct browser-to-portal call. Each boundary exists to keep secrets, raw responses, retries, data transformation and accountability under platform control.

4 Network and deployment topology

A production deployment should use a private network with public ingress only at the edge. The API, workers and data services should not be directly reachable from the internet.

SUBNET / SEGMENT	ALLOWED INBOUND	ALLOWED OUTBOUND	DEPLOYMENT NOTES
Public ingress	443 from approved user networks or public internet as policy requires	Only to frontend/API targets	WAF, TLS termination, DDoS and request limits
Private app	From load balancer and internal service identities	Database, queue, object storage, egress gateway	No public IP; container or VM workload
Private worker	From queue; restricted operator access	Approved source domains/IPs through egress gateway; object storage	Separate worker identity; sandbox document processing
Private data	From API/worker identities only	Backup service and key management endpoints	No direct user access; encryption and retention controls
Management	VPN/bastion and CI/CD identities	Monitoring, registry, KMS, deployment endpoints	MFA, short-lived credentials, break-glass logging

Server choices

For the hackathon, Docker Compose is sufficient: one Express API, one worker, Redis, MongoDB, MinIO and a reverse proxy. For a pilot, move to Kubernetes or a managed container platform only when there is a real availability and operations need. The security properties come from network isolation, identity and data controls, not from Kubernetes alone.

ENVIRONMENT	PURPOSE	MINIMUM CONTROLS
Local	Developer speed and mock adapters	Synthetic data only; no production credentials
CI	Automated tests and security checks	Ephemeral services; dependency and secret scanning
Staging	Provider sandbox and user acceptance	Separate accounts, masked data, source mocks/sandboxes
Production / pilot	Real bidder cases and approved integrations	Private network, managed secrets/KMS, backups, monitoring, incident response

5 Integration model for 10 or more platforms

Do not build ten bespoke workflows into the core. Build one adapter contract and implement each source behind it. The core consumes normalised verification results, not portal-specific response shapes.

ADAPTER CONTRACT	REQUIRED FIELDS	WHY IT MATTERS
Identity	adapter_id, provider, environment, version, owner	Makes every integration attributable and versioned
Authorisation	credential_ref, allowed_scopes, purpose_codes, consent_required	Prevents accidental reuse of a credential for a different purpose
Request	subject_type, subject_ref, case_id, correlation_id, requested_fields	Limits data to the exact check
Response	provider_status, normalized_result, raw_ref, received_at, source_timestamp	Separates source response from internal interpretation
Freshness	fresh_until, recheck_policy, stale_reason	Prevents stale “active” or “valid” status from silently passing
Failure	timeout, rate_limited, denied, unavailable, schema_changed	Turns failures into visible states and actionable operations
Audit	request_hash, response_hash, actor, policy_decision, timestamps	Supports replay, dispute and incident review

6 Source integration matrix

The matrix below is an architecture planning view. It distinguishes what the product wants to verify from how the source may actually be available. Production integration requires provider approval and a source-specific legal and technical review.

SOURCE / PLATFORM	USE IN BID VERIFICATION	INTEGRATION MODE	DATA RULE
GeM / bid source	Tender, corrigenda, ATC/STC, bid metadata	Approved GeM/portal integration or controlled document import	Store tender version and checksum; never assume current page state
API Setu	Discovery and approved access to government APIs	API Setu consumer onboarding, provider approval, sandbox first	Credential and consent records; respect rate limits and terms
GSTN	GSTIN existence, legal name, status, selected compliance signals	Official taxpayer search or approved service API	Store minimum normalized status; do not copy full returns unless required
Udyam / MSME	Registration number, enterprise category, QR/certificate match	Official portal verification or approved API	Effective-date rule for classification; preserve QR/document evidence
PAN / Income Tax	Identity match and tender-specific tax evidence	Approved verification channel or bidder-provided document	Mask PAN in UI and logs; do not infer tax compliance from PAN alone
MCA21	Company identity, status, directors or incorporation data where required	Approved MCA service or controlled evidence	Request only fields required by the tender clause
DigiLocker	Issuer-backed document retrieval and verification	Consent-led document flow and approved API	Record consent scope, issuer, URI/identifier and verification time
EPFO / ESIC	Labour compliance where applicable	Approved provider channel or official evidence	Make applicability explicit; distinguish unavailable from non-compliant
Startup India / DPIIT	Recognition status and certificate match	Approved API or document verification	Check entity identity, recognition number and effective date
NSIC	Registration and scheme eligibility	Approved source or evidence document	Do not treat a website screenshot as authoritative
BIS	Product or licence status where relevant	Approved verification endpoint or certificate validation	Bind certificate to product, issuer and validity period
Blacklisting / debarment sources	Tender-relevant exclusion checks	Authoritative published lists, notices or department source	Keep source snapshot and effective date; false positives require review
Make in India / local content	Declaration and supporting calculation	Bidder evidence plus tender-specific rule	Do not calculate local content without a defined rule and evidence

7 Consent, identity and purpose controls

API access and privacy are coupled. Before a sensitive verification call, BidSure should make a policy decision that combines the actor, case, purpose, requested fields, source terms and consent or authority basis.

CONTROL	IMPLEMENTATION
Institution identity	Use OIDC/SAML with the department or institution identity provider. Avoid building password authentication for the MVP unless unavoidable.
Roles	Bidder, procurement officer, reviewer, auditor, integration operator, security operator and system administrator.
Case scope	Every case and document carries an owning organisation, tender and role policy. A user must pass both role and case authorization.
Purpose code	Examples: TENDER_ELIGIBILITY, DOCUMENT_AUTHENTICITY, MSE_PREFERENCE, LABOUR_COMPLIANCE. A source adapter rejects unknown purposes.
Consent / authority record	Store subject, purpose, fields, source, actor, timestamp, notice version, consent/authority reference and expiry.
Credential isolation	Separate client IDs or API keys by source, environment and purpose where the provider supports it. Never place secrets in source code or frontend bundles.
Step-up control	Sensitive or high-impact checks require officer confirmation or second reviewer before execution.
Break-glass	Emergency access is time-limited, reason-coded, approved and automatically audited.

Important distinction

A bidder's consent to verify a document does not automatically authorise every possible lookup. Consent and authority must be scoped to the fields and purpose needed for the tender. API Setu terms place responsibility on the consumer to obtain appropriate consent and comply with provider requirements. [R3, R4]

8 Data classification and lifecycle

The safest data is data BidSure does not collect. The platform should store normalized verification evidence by default and retain raw source responses or original documents only when a policy, dispute or audit requirement needs them.

CLASS	EXAMPLES	DEFAULT HANDLING
Public	Tender title, public source URL, public rule text	Normal storage; integrity still matters
Internal	Case IDs, requirement text, operational metadata	Role-based access; normal encrypted storage
Confidential business	Turnover, pricing, bank or commercial evidence	Field-level access; encryption; no model training; short retention
Personal / sensitive	PAN, Aadhaar-linked data, signatory identity, employment details	Minimise, mask, encrypt, purpose-limit and restrict export
Security secret	API keys, private keys, refresh tokens, KMS material	Secret manager only; never log or persist with case data
Audit evidence	Hash, source timestamp, rule version, officer decision	Append-only store; tamper-evident export and controlled retention
LIFECYCLE STAGE	CONTROL	
Collect	Capture only fields required by the rule; validate file type and source scope.	
Process	Keep raw evidence separate from extracted fields; redact before logging or sending to an external model.	
Use	Display masked values by default; require a reason for reveal and log the reveal.	
Share	Share only normalized results and evidence references across modules; never fan out raw documents.	
Retain	Retention is configured by document type, tender status, dispute hold and legal requirement.	
Delete	Delete raw objects and derived copies together; preserve only the minimum audit record required by policy.	

9 Encryption, keys and secrets

Encryption is necessary but not sufficient. The key design question is who can decrypt, for what purpose, and whether the application can operate without exposing plaintext to every operator.

AREA	RECOMMENDATION	IMPLEMENTATION
Transport	TLS 1.2+ externally; mTLS where the provider requires or supports it	Reverse proxy, Node HTTPS client configuration, certificate rotation
Database	Encryption at rest and TLS connections; encrypt especially sensitive fields at application level	MongoDB encryption features or managed service; client-side field encryption for selected fields
Object storage	Server-side encryption with customer-managed keys if available; private buckets	S3-compatible storage with short-lived signed URLs
Secrets	No .env secrets in production; no secrets in logs or error payloads	Vault, cloud Secrets Manager or KMS-backed secret store
Key management	Separate keys by environment and data class; rotate and revoke	KMS policy, key rotation schedule and access review
Backups	Encrypted, access restricted and regularly restore-tested	Immutable or versioned backups; recovery key procedure
Exports	Redact or mask by default; watermark and record purpose	Officer report export policy and audit event

MongoDB note

MongoDB documents support TLS, encryption at rest and client-side or queryable encryption options. Choose the deployment edition and feature set only after confirming the data residency, key custody and operational requirements of the target organisation. [R8]

10 Recommended technology stack

The stack below is intentionally boring, typed and replaceable. It avoids adding microservices before the product has stable contracts.

LAYER	RECOMMENDED CHOICE	WHY
Runtime	Node.js 24 LTS for production; pin exact patch in CI	Use an actively supported LTS line. Node.js guidance says production applications should use Active or Maintenance LTS, not EOL releases. [R1]
Language	TypeScript 5.x with strict mode	Shared types, safer contracts and maintainable adapters
HTTP server	Express 5.x	Small middleware model, large ecosystem and easy operational control
Validation	Zod for request/domain schemas; zod-to-openapi or OpenAPI generator	Reject malformed inputs and keep API contracts explicit
Security middleware	helmet, cors with explicit allowlist, express-rate-limit, cookie-parser only if needed	Baseline headers, origin control and abuse limits. Express recommends Helmet for security headers. [R9]
Logging	pino and pino-http with redaction	Structured logs without secrets or raw personal data
HTTP clients	Node built-in fetch / undici, p-retry, bottleneck, opossum	Timeouts, retries, rate limits and circuit breakers per adapter
Authentication	OIDC/SAML integration with institutional IdP; jose for JWT validation	Avoid local password store; validate issuer, audience, expiry and nonce
Queue	BullMQ + Redis	OCR, adapter calls, retries and scheduled re-verification. BullMQ is a Node.js queue library built on Redis. [R7]
Database	MongoDB official Node driver; optional Mongoose only if the team needs ODM ergonomics	Flexible evidence records with explicit schema validation
Object storage	S3-compatible storage; @aws-sdk/client-s3 or MinIO client	Private documents, versioning and short-lived signed URLs
OCR and files	pdfjs-dist, sharp, file-type, isolated Tesseract service, ClamAV sidecar	Keep untrusted file processing away from the API process
Observability	OpenTelemetry, prom-client, pino transport to central log store	Trace one case across API, queue, adapter and decision
Testing	Vitest, Supertest, Playwright, Testcontainers, npm audit, OSV-Scanner	Unit, API, browser, integration and dependency security coverage

11 Express application structure

Use a modular monolith with clear internal boundaries. The first production deployment can run one API process and one worker process, but the code should behave as if these are separate trust domains.

MODULE	RESPONSIBILITY	DOES NOT DO
auth	Validate identity, session and role claims	Does not decide source-specific consent
policy	Evaluate role, case, purpose, fields and source permission	Does not fetch data
tender	Manage tender versions, clauses and corrigenda	Does not interpret source responses
documents	Upload, scan, hash, classify and link documents	Does not make eligibility decisions
adapters	Call one external source under one contract	Does not expose raw provider shape to the UI
verification	Normalize responses, freshness and cross-source checks	Does not alter source evidence
rules	Apply versioned deterministic rule definitions	Does not call external APIs
risk	Aggregate findings and create recommendation	Does not qualify or disqualify
decision	Capture officer outcome and reason	Does not allow silent overwrite
audit	Append events and export history	Does not become a mutable status table

Suggested repository layout

apps/api Express routes and middleware | **apps/worker** BullMQ processors | **packages/contracts** Zod and OpenAPI schemas | **packages/adapters** source clients | **packages/rules** versioned rules | **packages/security** masking, crypto and policy helpers | **infra** Docker, deployment and environment policy | **tests** unit, integration, security and replay fixtures

12 AI, OCR and document security

AI should help the system read and organize evidence. It should not become an unbounded decision-maker or an uncontrolled data export channel.

STEP	CONTROLLED DESIGN	OUTPUT
File intake	Allowlist PDF/images, enforce size/page limits, scan, hash and store original	Safe original object + checksum
OCR	Run in isolated worker; capture page and bounding box; record engine/version	Text fragments with confidence
Extraction	Use schema-constrained model output; treat document instructions as untrusted data	Candidate fields and citations to page/region
Cross-check	Compare fields across documents and source responses with deterministic normalizers	Match, mismatch, missing or review state
Rule evaluation	Versioned TypeScript rules evaluate structured evidence and freshness	Clause-level result
Recommendation	LLM may summarize findings and propose clarification questions	Advisory explanation linked to evidence
Human review	Officer approves correction, clarification or final decision	Immutable decision event

Model privacy rule

Do not send raw bidder documents to a third-party model unless the authority, data-processing terms, location, retention and purpose are approved. Prefer self-hosted or approved in-country inference for sensitive production data. For the SIH demo, use synthetic documents or redacted samples and label the model path.

13 Compliance, audit and explainability

The audit trail is a security control and a procurement control. It must answer who accessed what, why, from which source, under which policy, using which rule version, with which evidence, and what the officer decided.

EVENT	MINIMUM RECORD
Access	actor_id, role, case_id, purpose, fields viewed, timestamp, client context
Source request	adapter, credential_ref, policy decision, request hash, correlation ID, requested fields
Source response	provider status, response hash, source timestamp, normalized result, raw reference, freshness
Document action	upload, download, reveal, OCR, correction, export, deletion, actor and reason
Rule action	rule ID/version, inputs, evidence references, result, severity, engine version
Decision	officer, outcome, reason, unresolved findings, input snapshot hash, signed time
Admin action	credential change, role change, rule publish, retention override, break-glass reason

Tamper evidence

Use append-only audit events, event chaining or a signed daily digest. The goal is to make unauthorised editing detectable. An “immutable database” claim is not enough unless the platform also controls who can delete, alter, export or re-index the audit store.

14 Failure and availability architecture

The system must stay safe when external systems are unreliable. A source outage must not become an automatic pass, and a retry storm must not become a denial-of-service attack against a government platform.

FAILURE	SYSTEM STATE	MITIGATION
Timeout	Pending Re-verification	Short timeout, bounded retry, exponential backoff and visible freshness
Rate limit	Pending with retry-after	Bottleneck per adapter, queue backpressure and provider quota dashboard
Auth denied	Integration Blocked	Do not retry blindly; rotate or re-authorise under operator workflow
Schema changed	Review Required	Contract test, response quarantine and adapter version rollback
Conflicting sources	Review Required	Preserve both evidence records; do not pick one silently
OCR low confidence	Review Required	Manual correction with page-level evidence
Duplicate document	Review Required	Hash match and case history; never infer fraud automatically
Worker compromise	Job quarantined	Kill worker, revoke job token, preserve event, investigate and replay from clean input
Database unavailable	Read-only or degraded mode	No new final decisions; queue safe work; recover from backup and verify audit

15 Secure development and verification plan

Use OWASP ASVS 5.0 as the application security baseline and turn its requirements into testable acceptance criteria. ASVS is intended for security verification, developer guidance and procurement requirements. [R6]

GATE	CHECKS	PASS CRITERION
Design	Data inventory, threat model, source authority review, trust boundaries	No unowned data flow or credential path
Code	TypeScript strict, review, dependency pinning, secret scan, lint and unit tests	No critical findings; sensitive code has an owner
API	Schema validation, authz tests, rate limits, SSRF tests, file upload tests	Unauthorised access and unsafe URLs fail closed
Integration	Adapter contract tests, mock and sandbox replay, rate-limit simulation	Every adapter handles success, timeout, denied and schema change
AI	Prompt injection, malformed output, leakage and bias tests	Model cannot directly execute a policy or call a source
Runtime	Container hardening, least privilege, egress allowlist, image scan	No public data service; only approved routes
Audit	Replay case from stored evidence and verify decision hash	Same inputs and rule version reproduce the result
Release	Security sign-off, rollback plan, restore test and incident contact	Release owner accepts residual risk

Minimum test cases

- User from organisation A attempts to read organisation B case.
- Adapter receives a bidder-controlled URL and attempts SSRF.
- A PDF contains prompt-like text instructing the model to ignore policy.
- A portal returns ACTIVE with an old source timestamp.
- A source returns a different legal name from the uploaded certificate.
- A worker job is replayed after a timeout.
- An officer exports a report containing masked and unmasked fields.
- A revoked credential is still present in a worker process.

16 Implementation roadmap

Build the security boundary and evidence contract before adding all ten integrations. The fastest safe route is to prove one complete path, then replicate the adapter pattern.

PHASE	DELIVERABLES	EXIT CRITERION
0. Policy and scope	Data inventory, source authority matrix, consent/authority fields, synthetic test pack	Team agrees what each source is allowed to provide
1. Secure foundation	Node.js/Express API, OIDC/RBAC, private storage, secret management, audit events	No browser-to-source calls; access events are visible
2. Evidence pipeline	Upload, scan, hash, object storage, OCR worker, structured extraction	A reviewer can trace a field to page and checksum
3. Adapter framework	Common adapter contract, mock API Setu, GST and Udyam adapters	Success and failure states are consistent
4. Rule and decision engine	Tender versioning, requirement editor, deterministic rules, officer decision	Same evidence and rule version reproduce same result
5. Source expansion	MCA21, DigiLocker, Startup India, NSIC, EPFO/ESIC, BIS and debarment sources	Each source has owner, approval, scopes, freshness and runbook
6. Pilot hardening	Threat model, ASVS verification, penetration test, restore drill, DPIA/legal review	Pilot go/no-go decision with residual-risk register

Team ownership

ROLE	OWNS
Platform lead	Network, deployment, secrets, observability and incident response
Integration lead	Adapter contract, source onboarding, provider terms and connector tests
Document/AI lead	Safe file processing, OCR, extraction, model policy and evaluation
Rules lead	Tender taxonomy, rule registry, effective dates and test fixtures
Product/security lead	Threat model, data inventory, RBAC, audit, privacy review and officer workflow

17 Decisions the team should freeze now

- Use Node.js 24 LTS and TypeScript in strict mode for the API and workers.
- Use Express 5.x with a modular monolith, not microservices, for the SIH MVP.
- Use MongoDB for case/evidence metadata, private S3-compatible object storage for originals, and Redis + BullMQ for asynchronous work.
- Use OIDC/SAML with institutional identity where possible. Do not build a custom password system for the demo.
- Use mock or sandbox connectors and synthetic data until each real provider approves access.
- Make every source call pass a purpose and authority check before it leaves the private network.
- Treat AI output as candidate evidence or explanation. Deterministic rules and the Procurement Officer control the decision.
- Define retention, deletion, source freshness and audit export policies before collecting real bidder data.
- Use OWASP ASVS 5.0 as the security acceptance baseline and document residual risks.

What the architecture prevents

It prevents a frontend token leak from becoming a government portal breach, prevents one integration failure from producing a silent pass, prevents an LLM from becoming the final decision-maker, and makes a later source replacement possible without rewriting the compliance engine.

Appendix A Adapter response contract

This is a conceptual contract for the team to implement as a Zod schema and publish through OpenAPI. Provider-specific payloads remain inside the adapter.

FIELD	EXAMPLE	RULE
case_id	CASE-2026-001	Required on every request and response
adapter_id	gst.taxpayer-status.v1	Immutable identifier and version
purpose	TENDER_ELIGIBILITY	Must be allowlisted for the adapter
subject	{ type: "GSTIN", value_ref: "encrypted:..." }	Never log raw sensitive identifiers
status	VERIFIED PENDING REVIEW_REQUIRED FAILED	No boolean-only result
normalized_result	{ registrationStatus: "ACTIVE", legalNameMatch: true }	Only fields required by the rule
source_timestamp	2026-09-13T10:10:00Z	Provider timestamp if available
fresh_until	2026-09-20T10:10:00Z	Calculated by source policy
evidence_ref	EV-00045	Links to stored response/document evidence
request_hash / response_hash	sha256:...	Supports replay and tamper detection
audit_event_id	AUD-00077	Links to actor and policy decision

Appendix B Risk register

RISK	LEVEL	MITIGATION	OWNER
Provider access is not approved	High	Mock/sandbox adapter, formal onboarding and source capability registry	Integration lead
Sensitive data leaves India or approved boundary	High	Data-location review, self-hosted/approved model, egress policy and contract check	Security lead
Credential leak	High	Secret manager, per-source scope, rotation, mTLS where available and no logs	Platform lead
AI prompt injection or hallucination	High	Treat documents as data, constrained output, deterministic rules and review	AI lead
Portal outage or rate limit	Medium	Queue, bounded retry, circuit breaker, Pending state and operator runbook	Integration lead
False positive debarment/fraud result	High	Authoritative source, effective dates, evidence view and human escalation	Rules lead
Insider overreach	High	Case-level authz, purpose check, masked UI, reveal logging and dual control	Security lead
Audit record modified or deleted	High	Append-only store, signed digest, retention lock and restore drill	Platform lead
Unscanned malicious file	Medium	Allowlist, antivirus sidecar, sandboxed workers and size limits	Document lead

Appendix C Official references

The following official sources informed the security, integration and technology recommendations. They are reference points, not a substitute for provider approval, legal review or security accreditation.

R1 Node.js release and LTS policy

<https://nodejs.org/en/about/previous-releases>

R2 API Setu overview and integration flow

<https://docs.apisetu.gov.in/document-central/explore-apisetu/Overview.html>

R3 API Setu Terms of Use

<https://docs.apisetu.gov.in/document-central/terms-of-use/>

R4 API Setu access restrictions and consent responsibility

<https://docs.apisetu.gov.in/document-central/terms-of-use/Access%20Restrictions.html>

R5 MeitY Digital Personal Data Protection Rules 2025

<https://www.meity.gov.in/documents/act-and-policies/digital-personal-data-protection-rules-2025-gDOxUjMtQWa?pageTitle=Digital-Personal-Data-Protection-Rules-2025>

R6 OWASP ASVS 5.0

<https://owasp.org/www-project-application-security-verification-standard/>

R7 BullMQ documentation

<https://docs.bullmq.io/>

R8 MongoDB encryption documentation

<https://www.mongodb.com/docs/manual/core/security-data-encryption/>

R9 Express production security guidance

<https://expressjs.com/en/advanced/best-practice-security/>

R10 OpenAPI Specification

<https://swagger.io/specification/>

R11 General Financial Rules 2017

<https://doe.gov.in/files/inline-documents/GFR2017.pdf>

R12 GeM General Terms and Conditions 4.0

https://assets-bg.gem.gov.in/resources/upload/shared_doc/gtc/general-te-1675401798.pdf

R13 API Setu SOP supplied with project references

<https://cdn.apisetu.gov.in/portal/assets/SOP-APISETU.pdf>

End of architecture plan